LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK
PERTEMUAN 7
AUTOMATION TESTING

Disusun oleh:

Nama	: Januarsyah Akbar
NIM	: 24/535846/SV/24314
Kelas	: PL4B2
Dosen Pengampu	: Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

April 21, 2026

Daftar Isi

Daftar Isi **2**

Tujuan Praktikum **3**

Hasil dan Pembahasan **4**
2.1 Latihan 4
2.1.1 Pengujian Navigasi Google (`GoogleSafariTest.java`) 4
2.2 Tugas 7
2.2.1 Pengujian Login SauceDemo Sederhana 7
2.2.2 Pengujian Skenario Lengkap Pengguna SauceDemo 9

Kesimpulan **13**

Tujuan Praktikum

1. Mahasiswa mampu memahami kakas bantu pengujian antarmuka.
2. Mahasiswa mampu mengimplementasikan kode dasar pengujian antarmuka menggunakan kakas bantu.

Hasil dan Pembahasan

2.1 Latihan

Berikut adalah penjelasan dan langkah-langkah praktikum terkait latihan navigasi dan pengambilan informasi awal *browser* melalui pengujian menggunakan *Selenium Web Driver* pada pencarian Google.

2.1.1 Pengujian Navigasi Google (`GoogleSafariTest.java`)

Pada bagian latihan ini, kita membuat skenario di dalam `GoogleSafariTest.java` yang fokus utamanya adalah memvalidasi judul (*title*) situs dan berinteraksi secara sederhana dengan URL menggunakan SafariWebDriver.

1. Mendefinisikan Package dan Import

Tahapan pertama mencakup *import library* JUnit dan Selenium *WebDriver*.

```java
package org.example;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.safari.SafariDriver;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

Listing .1: Import Code Navigasi Google

2. Persiapan Awal WebDriver

Mirip dengan pengujian sebelumnya, kita harus menginisiasi *driver* menggunakan anotasi `@BeforeEach` untuk eksekusi secara berulang pada Safari.

```java
public class GoogleSafariTest {

    private WebDriver driver;

    @BeforeEach
    public void setUp() {
        driver = new SafariDriver();
    }
```

Listing .2: Inisialisasi Setup

3. Eksekusi Test Title Halaman

Pada *core test* di bawah ini, halaman diarahkan ke `https://www.google.com`. Lalu, string berupa properti *title* dari situs web diambil menggunakan `driver.getTitle()`. Assert kemudian memverifikasi eksistensinya.

```
    @Test
    public void testGoogleTitle() {
        driver.get("https://www.google.com");

        String title = driver.getTitle();

        assertEquals("G", title);
    }
```

Listing .3: Test Method Google Title

4. Penutupan Sesi Driver

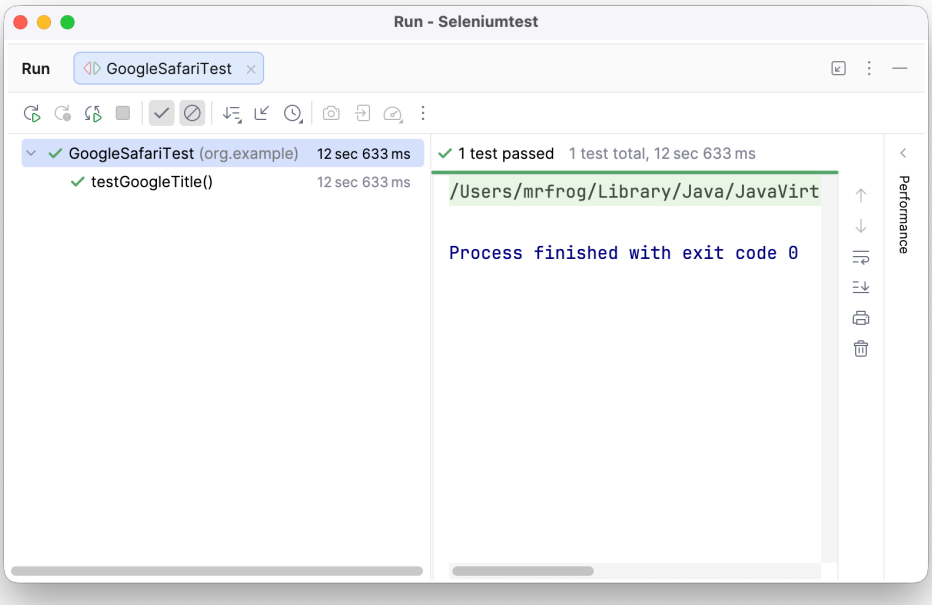
Proses teardown memanfaatkan anotasi `@AfterEach` akan melepas memori dan antarmuka dengan menterminasi Safari secara utuh.

```
    @AfterEach
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Listing .4: Teardown Browser GoogleSafariTest

Hasil Test

Berikut adalah tangkapan layar yang menunjukkan bahwa proses otomatisasi pencarian Google telah selesai dieksekusi oleh Selenium Webdriver.



Gambar 1: Hasil Eksekusi Test GoogleSafariTest

2.2 Tugas

Berikut adalah penjelasan dan tahap praktikum untuk pembuatan dan eksekusi *Automated Testing* terhadap interaksi aplikasi spesifik, yakni pendaftaran dan verifikasi autentikasi portal SauceDemo.

2.2.1 Pengujian Login SauceDemo Sederhana

Pada bagian ini, kita akan membuat kelas `SauceDemoLoginTest` untuk memverifikasi proses *login* untuk *standard user* pada situs SauceDemo.

1. Mendefinisikan Package dan Import

Langkah pertama adalah mendefinisikan nama *package* dan melakukan *import library* yang dibutuhkan, seperti *library* dari *framework* pengujian (JUnit 5) dan *library* Selenium WebDriver:

```java
package org.example;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.safari.SafariDriver;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

Listing .5: Import Dependencies

2. Persiapan WebDriver (Setup)

Selanjutnya, kita mendefinisikan kelas pengujian dan melakukan persiapan (*setup*) awal menggunakan anotasi `@BeforeEach`. Fungsi ini akan dieksekusi sebelum setiap *test method* untuk melakukan inisialisasi *browser* yang akan kita gunakan (dalam hal ini menggunakan `SafariDriver`).

```java
public class SauceDemoLoginTest {

    private WebDriver driver;

    @BeforeEach
    public void setUp() {
        driver = new SafariDriver();
    }
```

Listing .6: Inisialisasi Kelas dan Setup WebDriver

3. Skenario Pengujian Login (Test)

Tahap ini merupakan *core* pengujian, menggunakan anotasi `@Test`. WebDriver akan diinstruksikan untuk membuka halaman web (menggunakan `driver.get()`), mencari elemen berdasarkan ID (menggunakan `driver.findElement(By.id())`), lalu

melakukan interaksi masukan berupa pengetikan *keys* dan klik. Pada akhirnya *assertion* memastikan bahwa halaman yang terbuka setelah proses *login* adalah halaman *inventory*.

```
    @Test
    public void testLogin() {
        driver.get("https://www.saucedemo.com/");

        WebElement usernameInput = driver.findElement(By.id("user-name"));
        usernameInput.sendKeys("standard_user");

        WebElement passwordInput = driver.findElement(By.id("password"));
        passwordInput.sendKeys("secret_sauce");

        WebElement loginButton = driver.findElement(By.id("login-button"));
        loginButton.click();

        String currentUrl = driver.getCurrentUrl();
        assertEquals("https://www.saucedemo.com/inventory.html", currentUrl);
    }
```

Listing .7: Test Method Login Sederhana

4. Membersihkan WebDriver (Teardown)

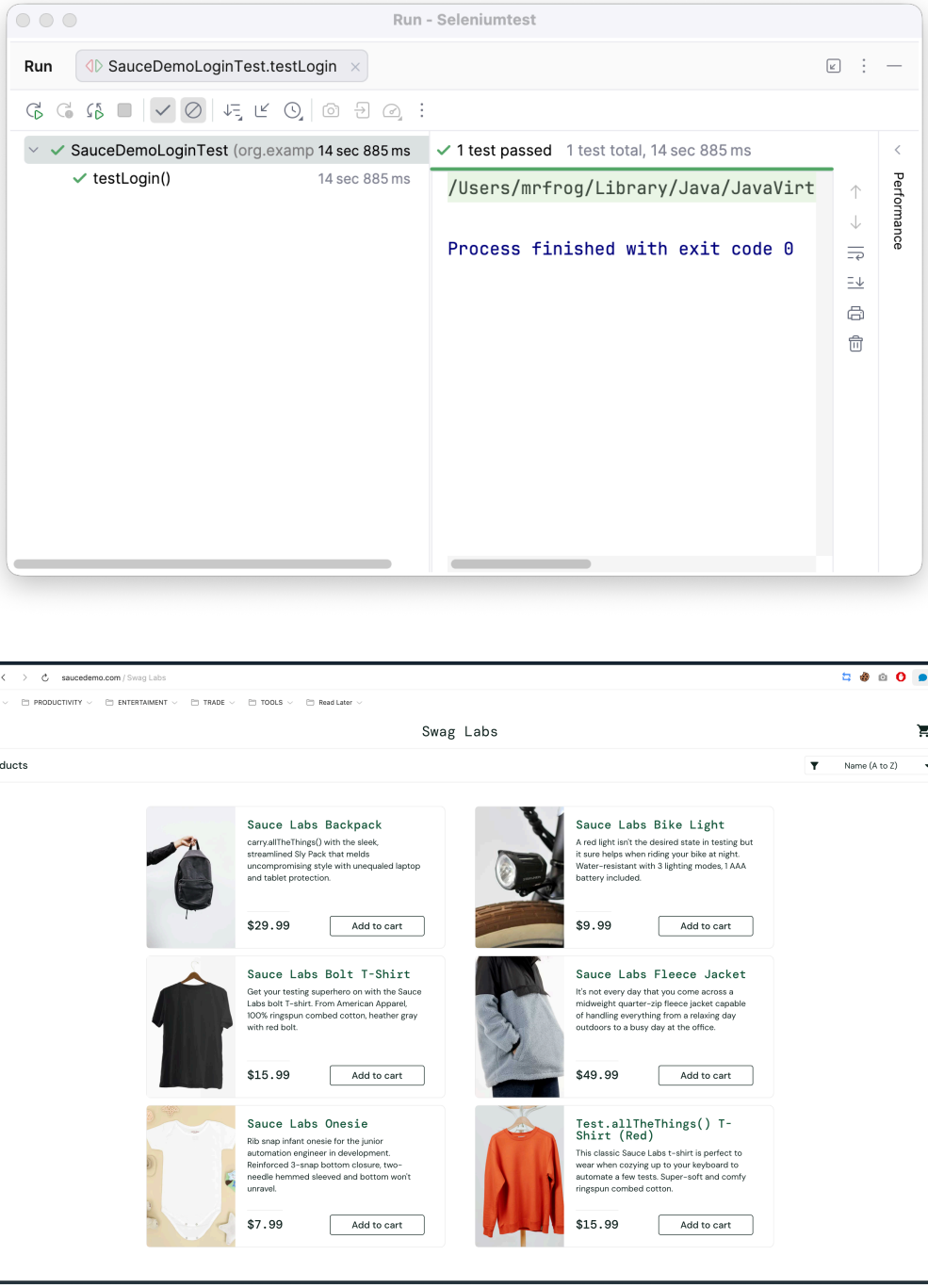
Bagian terakhir menggunakan anotasi `@AfterEach` bertujuan menutup sesi *browser* untuk menghemat *resource* sistem setiap kali sekuens pengujian dari suatu *test case* telah selesai dilaksanakan:

```
    @AfterEach
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Listing .8: Penutupan Sesi Browser

Hasil Test

Berikut dilampirkan tangkapan layar sebagai *output* yang membuktikan eksekusi *Automated Testing* telah berjalan dan berhasil mencapai (*passed*) skenario uji tersebut.



Gambar 2: Hasil Eksekusi Test Login SauceDemo (Test Runner dan Tampilan Browser Output)

2.2.2 Pengujian Skenario Lengkap Pengguna SauceDemo

Berikut adalah pengujian yang lebih ekstensif untuk berbagai jenis tipe pengguna (`locked_out_user`, `problem_user`, `performance_glitch_user`, dan lainnya) yang membutuhkan *WebDriverWait* untuk mencegah error karena *loading* elemen web.

1. Import Library dan Konfigurasi Awal

Kita menambahkan paket tambahan seperti `WebDriverWait` maupun `ExpectedConditions` untuk memfasilitasi penungguan eksplisit (*Explicit Wait*).

```java
package org.example;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.safari.SafariDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class SauceDemoAllUsersLoginTest {

    private WebDriver driver;
    private WebDriverWait wait;

    @BeforeEach
    public void setUp() {
        driver = new SafariDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }
```

Listing .9: Import dan Setup Skenario Lengkap

2. Pembuatan Helper Method Login

Alih-alih menulis sintaks Selenium identik di setiap skenario *user*, kita mengelompokkannya dalam sebuah *method helper* bernama `login` agar perancangan *Test Case* menjadi lebih rapi dan ringkas (*reusable step*).

```java
    private void login(String username, String password) {
        driver.get("https://www.saucedemo.com/");
        driver.findElement(By.id("user-name")).sendKeys(username);
        driver.findElement(By.id("password")).sendKeys(password);
        driver.findElement(By.id("login-button")).click();
    }
```

Listing .10: Method Helper Interaksi Login

3. Test Case Standar dan Blocked User

Pengujian di bawah ini dilakukan berturut-turut untuk validasi *standard_user* yang seharusnya sukses (diarahkan ke `/inventory.html`) serta validasi *locked_out_user* yang akan memunculkan pesan *error* khusus dan tidak mengubah URL.

```
    @Test
    public void testStandardUserLogin() {
        login("standard_user", "secret_sauce");
        wait.until(ExpectedConditions.urlToBe("https://www.saucedemo.com/inventory.html"));
        assertEquals("https://www.saucedemo.com/inventory.html", driver.getCurrentUrl());
    }

    @Test
    public void testLockedOutUserLogin() {
        login("locked_out_user", "secret_sauce");
        WebElement errorElement = wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("h3[data-test='error']")));
        assertTrue(errorElement.getText().contains("Epic sadface: Sorry, this user has been locked out."));
        assertEquals("https://www.saucedemo.com/", driver.getCurrentUrl());
    }
```

Listing .11: Pengujian Standard dan Blocked User

4. Uji Lanjutan dan Teardown

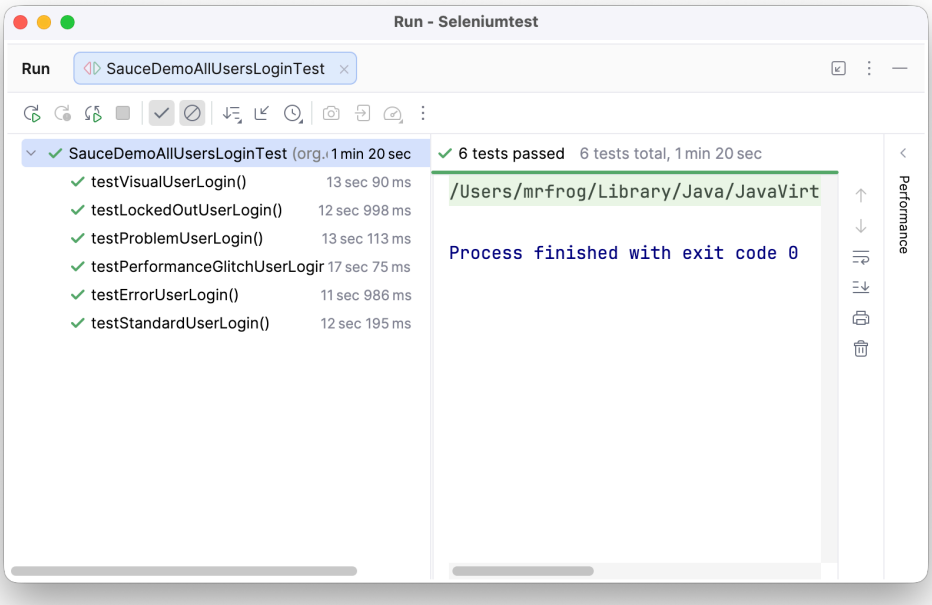
Sama halnya dengan kelas login dasar, setelah semua *test method* dijalankan, kita perlu menutup *driver browser*.

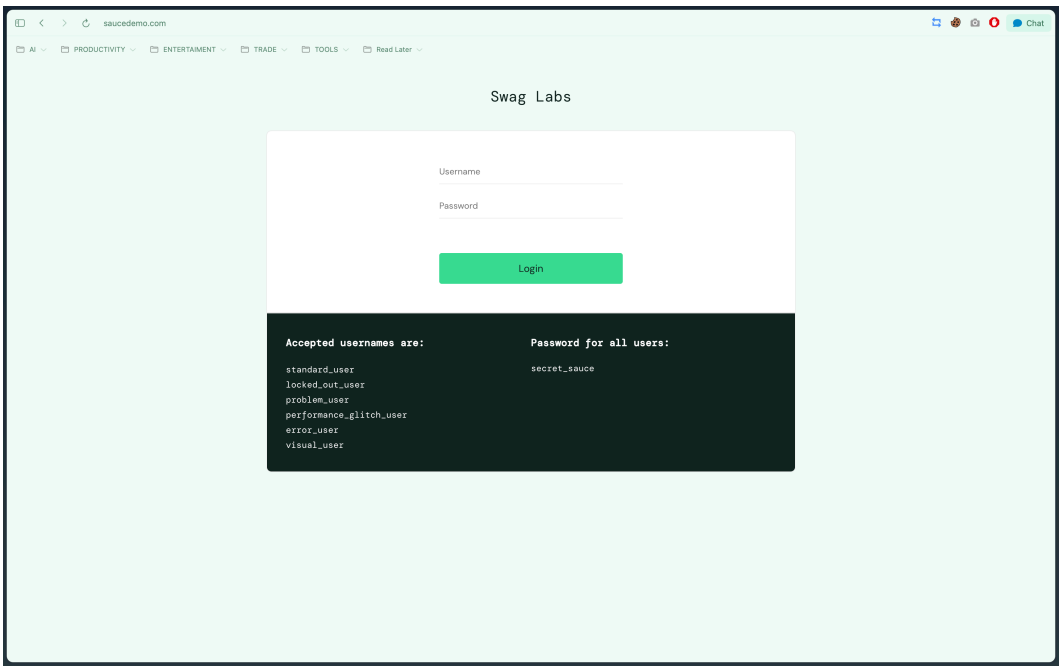
```
    @Test
    public void testProblemUserLogin() {
        login("problem_user", "secret_sauce");
        wait.until(ExpectedConditions.urlToBe("https://www.saucedemo.com/inventory.html"));
        assertEquals("https://www.saucedemo.com/inventory.html", driver.getCurrentUrl());
    }

    // Beberapa test lain untuk performance_glitch_user, error_user, dan visual_user juga berhasil diuji...

    @AfterEach
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Listing .12: Pengujian Tambahan dan Akhir Siklus

Hasil Test

Sama halnya dengan kelas sebelumnya, berikut kami tampilkan hasil *run* atau eksekusi *Test Suite* yang menandakan skenario berhasil dilakukan.





Gambar 3: Hasil Eksekusi Test Skenario Seluruh Pengguna (Test Runner dan Kondisi URL Hasil Tes)

https://github.com/januarsyah901/PPPL/tree/9d622d374dcf5c9e04c560607ba4e59049f3681a/Pertemuan%207

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan *kakas bantu* seperti Selenium WebDriver sangat mempermudah pembuatan *automated testing* atau pengujian otomatis antarmuka pada suatu aplikasi secara terstruktur. Pengujian *login* pada website `SauceDemo` dengan menggunakan berbagai *user* telah berhasil diotomatisasi secara rinci.

Selain itu, pendokumentasian skrip pengujian berbasis *Java* serta JUnit telah menunjukkan bagaimana *webdriver* diinstansiasi, dinavigasi mencari *element* dan divalidasi dengan struktur kode secara langsung. Ini sangat menghemat waktu dibandingkan dengan menguji suatu *form login* dan interaksi-interaksi web secara manual berulang-ulang, menstimulasi produktivitas untuk QA (Quality Assurance) serta *Developer* pada tahap-tahap integrasi perangkat lunak nantinya.